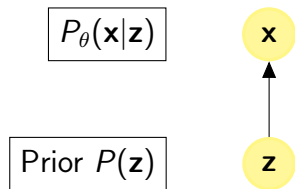


Variational Auto Encoders(VAEs)

A probabilistic approach for training F_θ which is assumed to model $P_\theta(\mathbf{x}|\mathbf{z})$. Thus, the underlying graphical model is:



During training we learn θ so as to maximize the likelihood of the training data.

Training VAEs

Training data $D = \{\mathbf{x}^i\}_{i=1}^N$ is observed and $\mathbf{z}$ is hidden. Marginalize over $\mathbf{z}$ to compute $P(\mathbf{x})$.

$$\max_{\theta} \sum_{i=1}^N \log P_{\theta}(\mathbf{x}^i) = \max_{\theta} \sum_{i=1}^N \log \int_{\mathbf{z}} P_{\theta}(\mathbf{x}|\mathbf{z}) P_{\theta}(\mathbf{z}) d\mathbf{z}$$

$\mathbf{z} \in R^k, \mathbf{x} \in R^n$ Integrating over all $\mathbf{z}$ is intractable. How to tackle this?

Use variational approximation of the objective.

Variational approximation

$$\begin{aligned} & \max_{\theta} \sum_{i=1}^N \log P_{\theta}(\mathbf{x}^i) \\ &= \max_{\theta} \sum_{i=1}^N \log \int_{\mathbf{z}} P_{\theta}(\mathbf{x}^i | \mathbf{z}) P_{\theta}(\mathbf{z}) d\mathbf{z} \\ &= \max_{\theta, q} \sum_{i=1}^N \int_{\mathbf{z}} \underbrace{q(\mathbf{z} | \mathbf{x}^i) \log P_{\theta}(\mathbf{x}^i | \mathbf{z})}_{\text{ } } - \underbrace{\int_{\mathbf{z}} q(\mathbf{z} | \mathbf{x}^i) \log \frac{q(\mathbf{z} | \mathbf{x}^i)}{P_{\theta}(\mathbf{z})} d\mathbf{z}}_{\text{ } } \end{aligned}$$

$F(\theta, q)$

$$\max_{\theta, q} F(\theta, q)$$

Challenges of solving the variational objective

EM algorithm: solve $\max_{\theta, q} F(\theta, q)$ directly

E-step

$$\max_q F(\theta^t, q) \Rightarrow \underline{q^t(z|x)} \equiv P(z|x, \theta^t) = \frac{P(x|z, \theta^t)P(z)}{\int_z P(x|z, \theta^t)P(z)}$$

M-step

$$\theta^{t+1} \leftarrow \operatorname{argmax}_{\theta} F(\theta, q^t)$$

$$\max_{\theta} \sum_{i=1}^N \int_z \frac{P(x^i|z, \theta^t)P(z)}{\int_{z^i} P(x^i|z^i, \theta^t)P(z^i)} P_{\theta}(x^i|z) \cdot q^t(z|x^i)$$



Challenges of solving the variational objective

$$\max_{\theta, q} \sum_{i=1}^N \int \underline{q(z|x^i)} \underline{P_{\theta}(x^i|z)} - \underbrace{\int \underline{q(z|x^i)} \log \frac{q(z|x^i)}{P(z)}}_{\text{KL}}$$

- ① daunting integral
- ② $q(z|x^i)$ is not tractably computable
- ③ even this looks hard.

Challenges of solving the variational objective

Training VAE: Trick I

To compute $q(z|x^i)$ use another neural network $q_\phi(z|x^i)$ with shared parameters ϕ

$$\max_{\theta} \sum_{i=1}^N \log P_{\theta}(\mathbf{x}^i)$$

$$q(z|x^i) \approx q_{\phi}(z|x^i)$$

$$= \max_{\theta, q} \sum_{i=1}^N \int_{\mathbf{z}} \underline{q(\mathbf{z}|\mathbf{x}^i)} \log P_{\theta}(\mathbf{x}^i|\mathbf{z}) - \int_{\mathbf{z}} q(\mathbf{z}|\mathbf{x}^i) \log \frac{q(\mathbf{z}|\mathbf{x}^i)}{P_{\theta}(\mathbf{z})} d\mathbf{z}$$

$$\geq \max_{\theta, \phi} \sum_{i=1}^N \underbrace{\int_{\mathbf{z}} (q_{\phi}(\mathbf{z}|\mathbf{x}^i) \log P_{\theta}(\mathbf{x}^i|\mathbf{z}))}_{\text{First term}} - \underbrace{\int_{\mathbf{z}} q_{\phi}(\mathbf{z}|\mathbf{x}^i) \log \frac{q_{\phi}(\mathbf{z}|\mathbf{x}^i)}{P_{\theta}(\mathbf{z})} d\mathbf{z}}_{\text{Second term}}$$

Training VAE: Trick II, Sample instead of integration

$$\sum_{i=1}^N \int_{\mathbf{z}} (q_{\phi}(\mathbf{z}|\mathbf{x}^i) \log P_{\theta}(\mathbf{x}^i|\mathbf{z})) d\mathbf{z} = \sum_{i=1}^N E_{q_{\phi}(\mathbf{z}|\mathbf{x}^i)} [\log P_{\theta}(\mathbf{x}^i|\mathbf{z})]$$
$$\approx \sum_{i=1}^N \frac{1}{r} \sum_{j=1}^r \log P_{\theta}(\mathbf{x}^i|\mathbf{z}_j)$$

Handwritten notes in pink:

- $E_{\mathbf{z} \sim q(\mathbf{z})} [G(\mathbf{z})] = \int q(\mathbf{z}) G(\mathbf{z}) d\mathbf{z}$
- $\approx \frac{1}{r} \sum_{j=1}^r G(\mathbf{z}_j)$
- $\mathbf{z}_1, \dots, \mathbf{z}_r \sim q(\mathbf{z})$

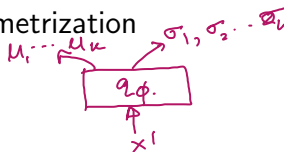
where $\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_r \sim q_{\phi}(\mathbf{z}|\mathbf{x}^i)$

How to sample from q_{ϕ} while still allowing the gradients to flow through parameters ϕ ?

Training VAE: Trick III, Reparameterize

Choose a form of $q(\mathbf{z}|\mathbf{x})$ that allows easy reparameterization

- Assume $q_\phi(\mathbf{z}|\mathbf{x}^i) = \prod_k q_\phi(z_k|\mathbf{x}^i)$
- Let $q_\phi(z_k|\mathbf{x}^i) \sim \mathcal{N}(\mu_{z_k|\mathbf{x}^i}, \sigma_{z_k|\mathbf{x}^i}^2)$
- To sample from $\mathcal{N}(\mu_{z|\mathbf{x}^i}, \sigma_{z|\mathbf{x}^i}^2)$, we use **Reparameterization trick**.
- Sample from a parameterless distribution. Here, we choose $\mathcal{N}(0, I)$. Let the samples be $v_1 \dots v_r$
- $z_j = v_j \sigma_{z|\mathbf{x}^i} + \mu_{z|\mathbf{x}^i}$ are samples from $\mathcal{N}(\mu_{z|\mathbf{x}^i}, \sigma_{z|\mathbf{x}^i})$.



Training VAE(Continued)

$$\sum_{i=1}^N \sum_{j=1, \mathbf{z}_j \sim q_\phi(\mathbf{z}_k | \mathbf{x}^i)}^r \log P_\theta(\mathbf{x}^i | \mathbf{z}_j) = \sum_{i=1}^N \sum_{j=1, \mathbf{v} \sim \mathcal{N}(0, I)}^r \log P_\theta(\mathbf{x}^i | \underbrace{\mu_{\mathbf{z} | \mathbf{x}^i} + \sigma_{\mathbf{z} | \mathbf{x}^i} \mathbf{v}_j}_{\text{circled in pink}})$$

$\mu_{z_1 | \mathbf{x}}, \dots, \mu_{z_K | \mathbf{x}}$

$\sigma_{z_1 | \mathbf{x}}, \dots, \sigma_{z_K | \mathbf{x}}$

$\mathbf{v}_1, \dots, \mathbf{v}_r$
 $\sim \mathcal{N}(0, \mathbf{I})$

Encoder q_ϕ

$q(\mathbf{z} | \mathbf{x}^i)$

$\mathbf{x}$

Training VAE: Trick IV, KL distance in closed form

$\int_{\mathbf{z}} q_{\phi}(\mathbf{z}|\mathbf{x}^i) \log \frac{q_{\phi}(\mathbf{z}|\mathbf{x}^i)}{P_{\theta}(\mathbf{z})}$ is the KL distance between $q_{\phi}(\mathbf{z}|\mathbf{x}^i)$ and $P_{\theta}(\mathbf{z})$.

KL distance in closed form

Assume $P_{\theta}(z) = \mathcal{N}(0, 1)$, $q(z|x^i) = \mathcal{N}(\mu_i, \sigma_i^2)$

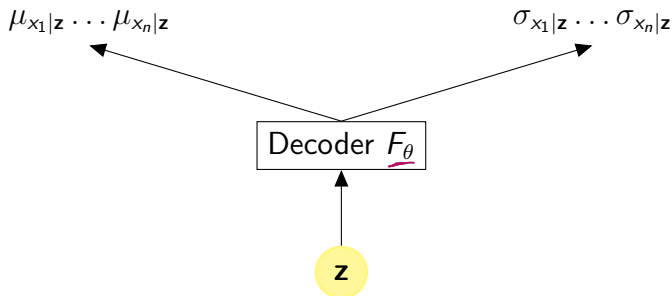
$$\begin{aligned} KL(\underline{q(z|x)}, \underline{P(z)}) &= \int_{\mathbf{z}} q(\mathbf{z}|\mathbf{x}) \log \frac{q(\mathbf{z}|\mathbf{x}^i)}{P(\mathbf{z})} \\ &= \int_{\mathbf{z}} \mathcal{N}(\mu_i, \sigma_i^2) 0.5 \left[-\frac{(z - \mu_i)^2}{\sigma_i^2} - \log \sigma_i^2 + z^2 \right] \\ &= 0.5 \left[-\frac{\sigma_i^2}{\sigma_i^2} - \log \sigma_i^2 + \int_{\mathbf{z}} z^2 \mathcal{N}(\mu_i, \sigma_i^2) \right] \\ &= \underline{0.5[-1.0 - \log \sigma_i^2 + \mu_i^2 + \sigma_i^2]} \end{aligned}$$

Representing $P_{\theta}(\mathbf{x}|\mathbf{z})$

Many forms of $P_{\theta}(\mathbf{x}|\mathbf{z})$ possible.

One possible choice to make it a conditional Gaussian distribution whose parameters are obtained from a deep network.

$$\underline{P(\mathbf{x}|\mathbf{z})} = \mathcal{N}(\underline{\mathbf{x}}; \underline{\mu} = [\underline{\mu_{x_1|\mathbf{z}}} \dots \mu_{x_n|\mathbf{z}}], \Sigma = \text{diagonal}[\underline{\sigma_{x_1|\mathbf{z}}} \dots \sigma_{x_n|\mathbf{z}}])$$



Representing $P_{\theta}(\mathbf{x}|\mathbf{z})$

When $\mathbf{x}$ is discrete..

Assume each $\mathbf{x} = [x_1, x_2, \dots, x_n]$

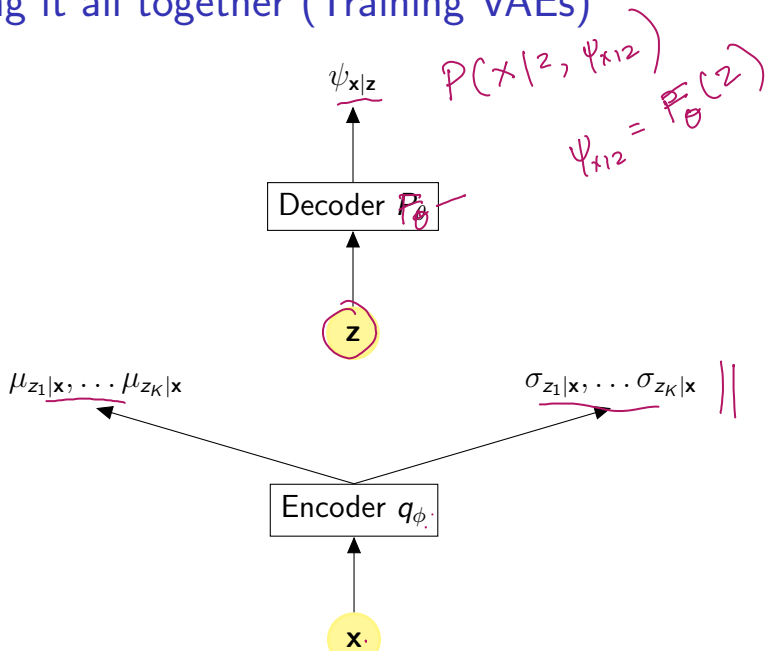
$$\begin{aligned} P_{\theta}(\mathbf{x}|\mathbf{z}) &= \prod_{j=1}^n P_{\theta}(x_j|\mathbf{z}) \\ &= \prod_{j=1}^n \eta_j^{x_j} (1-\eta_j)^{(1-x_j)} \end{aligned}$$

$$x_j \in \{0, 1\}$$



$\eta_j =$ probability of x_j being 1

Putting it all together (Training VAEs)



Overall Training Algorithm

Initialize θ and ϕ network parameters randomly.

for number of training iterations **do** -

Sample minibatch of B examples $\mathbf{x}^1, \mathbf{x}^2, \dots, \mathbf{x}^B$ from data D

Compute $\mu_{\mathbf{z}|\mathbf{x}^i}, \sigma_{\mathbf{z}|\mathbf{x}^i} \leftarrow q_{\phi}(\mathbf{z}|\mathbf{x}^i)$

Get Br samples $\mathbf{v}^1, \mathbf{v}^2, \dots, \mathbf{v}^{Br}$ from noise prior $P(\mathbf{z})$. Each is of K dims.

Compute $\mathbf{z}^{i,r} = \sigma_{\mathbf{z}|\mathbf{x}^i} \mathbf{v}^{i,r} + \mu_{\mathbf{z}|\mathbf{x}^i}$

Compute $\mu_{\mathbf{x}^i|\mathbf{z}}, \sigma_{\mathbf{x}^i|\mathbf{z}} \leftarrow P_{\theta}(\mathbf{z}^{i,r})$

loss(θ, ϕ) =

$$\sum_{i,r} -\frac{1}{r} \log N(\mathbf{x}^i | \mu_{\mathbf{x}^i|\mathbf{z}}, \sigma_{\mathbf{x}^i|\mathbf{z}}) + \sum_i \mu_{\mathbf{z}|\mathbf{x}^i}^2 + \sigma_{\mathbf{z}|\mathbf{x}^i}^2 - \log \sigma_{\mathbf{z}|\mathbf{x}^i}$$

Update θ, ϕ using gradients of loss(θ, ϕ)

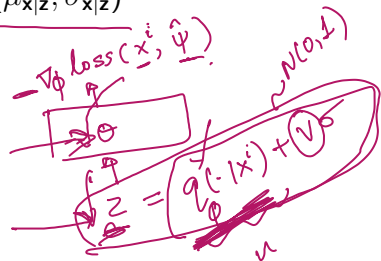
end for=0

Inference on Trained VAEs

During inference we can generate new samples $\mathbf{x}$ using the following three steps:

- 1 Sample a latent $\mathbf{z}$ from Gaussian prior $\mathbf{z} \sim \mathcal{N}(0, I_{k \times k})$
- 2 Pass that through the decoder to get mean and variance of the $\mathbf{x}$. $\mu_{\mathbf{x}|\mathbf{z}}, \sigma_{\mathbf{x}|\mathbf{z}} \leftarrow F_{\theta}(\mathbf{z})$
- 3 Sample a $\mathbf{x}$ from the Gaussian: $\mathbf{x} \sim \mathcal{N}(\mu_{\mathbf{x}|\mathbf{z}}, \sigma_{\mathbf{x}|\mathbf{z}})$

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial \theta}$$



Additional Resources

- Code Sample: <https://colab.research.google.com/github/keras-team/keras-io/blob/master/examples/generative/ipynb/vae.ipynb>
- Unsupervised Deep Learning, NeurIPS 2018 Tutorial, <https://nips.cc/Conferences/2018/Schedule?showEvent=10985>

Limitation of VAEs

- 1 Generation quality poor since it is difficult to train ϕ and θ jointly

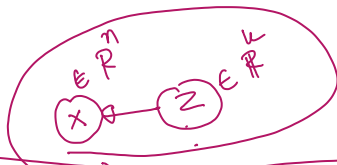
Types of Latent Variable Generative Models

Different ways of training D_θ

- 1 Variational Auto Encoders(VAEs)
- 2 Denoising Diffusion models 
- 3 Normalizing Flows (NF)
- 4 Generative Adversarial Networks(GANs)

Structured Variational Approximation

$$\max_{\theta} \sum_{i=1}^N \log P_{\theta}(\mathbf{x}^i)$$



$$= \max_{\theta} \sum_{i=1}^N \underbrace{\max_{q(\mathbf{z}|\mathbf{x}^i)} \int_{\mathbf{z}} (q(\mathbf{z}|\mathbf{x}^i) \log P_{\theta}(\mathbf{x}^i|\mathbf{z}))}_{\text{Expected Likelihood}} - \underbrace{\int_{\mathbf{z}} q(\mathbf{z}|\mathbf{x}^i) \log \frac{q(\mathbf{z}|\mathbf{x}^i)}{P_{\theta}(\mathbf{z})} d\mathbf{z}}_{\text{KL-distance}(q(\mathbf{z}|\mathbf{x}^i), P_{\theta}(\mathbf{z}))}$$

Assume that $P(\mathbf{x}|\mathbf{z})$ is further factorized as a chain Bayesian network as follows where $\mathbf{x} \mapsto \mathbf{x}_0$, $\mathbf{z} \mapsto \mathbf{x}_1, \dots, \mathbf{x}_T$

